

Guia de Deploy Local Hardhat — WEbdEX

Objetivo

Preparar ambiente local no Hardhat para testes e compreensão da estrutura do projeto WEbdEX.

Arquitetura Geral

O contrato central é o **WEbdEXFactoryV5**.

Ele registra, para cada bot (manager), os endereços de:

- manager
- strategy
- subAccount
- payments
- tokenPass
- network
- feeCollector
- owner
- prefix / name

Os demais contratos consultam essas informações via `getBotInfo(address)`.

Ordem Correta de Deploy

1. Deploy Tokens Mock

Criar mocks ERC20 para testes:

- MockUSDT (liquidez)
 - MockPOL (opcional / gas)
 - MockPASS (token de assinatura)
-

2. Deploy Factory

Deploy do contrato:

- `WEbdEXFactoryV5`

Salvar endereço do factory.

3. Deploy dos Módulos

Todos recebem `factory.address` no constructor.

Deploy:

1. `WEbdEXStrategiesV5(factory)`
2. `WEbdEXSubAccountsV5(factory)`
3. `WEbdEXPaymentsV5(factory)`
4. `WEbdEXNetworkV5(factory)`
5. `WEbdEXManagerV5(factory)`

Salvar todos os endereços.

4. Registrar Bot no Factory

Executar:

```
factory.addBot(  
    name,  
    prefix,  
    owner,  
    managerAddress,  
    strategyAddress,  
    subAccountAddress,  
    paymentsAddress,  
    tokenPassAddress,  
    networkAddress,  
    feeWithdrawNetwork,  
    feeCollectorNetworkAddress,  
    feeTiers  
)
```

Importante

A chave principal do sistema é:

```
managerAddress
```

Internamente:

```
bots[managerAddress] = config
```

Muitos contratos usam:

```
factory.getBotInfo(address(this))
```

5. Criar Estratégia

Após registrar o bot:

```
strategies.addStrategy(  
    "Conservative",  
    "CONS",  
    manager.address  
)
```

Consultar token criado:

```
strategies.getStrategies(manager.address)
```

6. Liberar Executor na Whitelist

Para chamar `openPosition()`:

```
payments.addAddressToWhitelist(  
    manager.address,  
    executor.address  
)
```

7. Registrar Usuário

Usuário executa:

```
manager.register(referrer, "Main")
```

Cria subaccount automaticamente.

8. Depositar Saldos

Gas

Se existir `gasAdd()` payable:

```
manager.gasAdd({ value: ... })
```

Pass Token

Mintar PASS ao usuário, aprovar e chamar:

```
manager.passAdd(amount)
```

Liquidez

Aprovar USDT e chamar:

```
manager.LiquidityAdd(  
  [accountId],  
  strategyToken,  
  usdt.address,  
  amount  
)
```

9. Abrir Posição

Executor autorizado chama:

```
payments.openPosition(...)
```

Resumo Rápido

1. Deploy Mock Tokens
2. Deploy Factory
3. Deploy Strategies
4. Deploy SubAccounts
5. Deploy Payments

```
6. Deploy Network
7. Deploy Manager
8. Factory.addBot(...)
9. Strategies.addStrategy(...)
10. Payments.addAddressToWhitelist(...)
11. User.register(...)
12. Deposits
13. openPosition()
```

Dependências Críticas

Manager

Consulta config via:

```
getBotInfo(address(this))
```

Payments

Depende de:

- strategyAddress
- managerAddress
- subAccountAddress

SubAccounts

Valida chamadas do Payments:

```
onlyPayments(manager)
```

Estrutura Recomendada de Scripts

```
scripts/
  deploy.js
  setup.js
  exploit-test.js
```

Pegadinha Principal

Se registrar o `managerAddress` errado no `addBot`, o sistema inteiro quebra.

Próximos Passos Possíveis

1. Script Hardhat completo
2. Mocks ERC20
3. Teste de exploit
4. Fluxo completo de openPosition
5. Auditoria de permissões